

A T G G C T A A A C C G T A A

BioCompiler 1.0

DNA Transcriber: de uma fita de DNA ao pré-mRNA

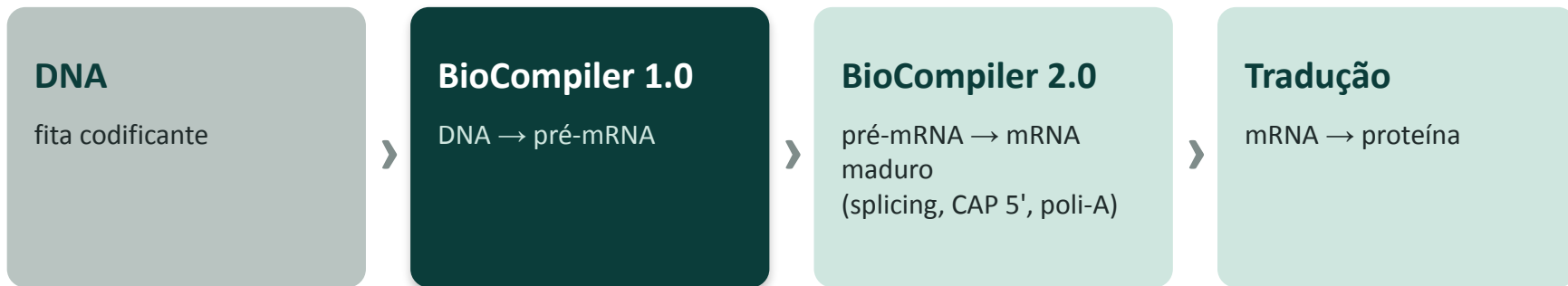
A U G G C U A A A C C G U A A

Gustavo Bastos Ribeiro Carvalho

João Vinicio Teles Rosal

Tópicos em Bioinformática · Ciência da Computação · UFPI

Onde o 1.0 entra no fluxo da informação genética



Este trabalho

Escopo da Fase I. O 1.0 termina no pré-mRNA: não faz maturação do RNA nem tradução. A meta é entregar um pré-mRNA válido e confiável para o 2.0 processar. A gramática usada é didática: representa conceitos reais da expressão gênica sem reproduzir toda a complexidade biológica.

Pensado como um compilador

1 Verificação léxica

Alfabeto do DNA: apenas A, T, C e G. Qualquer outro símbolo é um erro léxico.

BUG - base inválida

2 Análise sintática

A gramática de uma região codificante: START, trincas de códons, STOP em fase.

START ausente · frameshift ·
STOP ausente · nonsense

3 Gerar a sequência

Se a sequência é válida, o "código alvo" é o pré-mRNA: cada T vira U.

CORRETO + pré-mRNA

ATG . . . TAA / TAG / TGA

Região codificante válida segundo a convenção didática da atividade

Entrada e saída

Entrada

- Upload de um arquivo TXT (uma sequência por linha) ou CSV (coluna "entrada")
- Somente A, T, C, G maiúsculos, sem separadores
- Cada linha é validada de forma independente, na ordem em que aparece
- Sem assumir de antemão qual caso é

entrada.txt

```
CCCATGGCTAAACCGTAA
GGGATGCCCGGGTAG
TTTATGAAACCCGGGTGA
```

Saída, na interface web

Tabela + resumo por status, e um relatório detalhado por entrada

```
ENTRADA: 1
Sequência: ATGGCTAAACCGTAA
STATUS: CORRETO
pré-mRNA: AUGGCUAAACCGUAA
```

Dois botões de download

```
CSV: entrada,status,pre_mRNA
TXT: linha;status;resultado;pre_mRNA
1;OK;CORRETO;AUGGCUAAACCGUAA
```

Como a região codificante é identificada

Exemplo: linha 1 do arquivo de entrada da especificação (prefixo de 3 bases)



-  **Bases antes do START** permitidas — desde que seu total também seja múltiplo de 3
-  **START** primeiro ATG cuja posição, contada da 1ª base da linha, é múltipla de 3
-  **Códons internos** leitura em trincas a partir do START
-  **STOP** primeiro TAA, TAG ou TGA em fase; deve encerrar a linha

Transcrição

Só quando a sequência é válida. Cada timina vira uracila, ordem preservada.

DNA
ATGGCTAAACCGTAA

pré-mRNA
AUGGCUAAACCGUAA

Os seis casos que o programa reconhece

Caso	Entrada	Situação	Resposta esperada
1		Sequência válida	CORRETO
2		Caractere fora de A/T/C/G	BUG - base inválida
3		ATG de início ausente	BUG - START ausente
4		Nenhum STOP em fase após o START	BUG - STOP ausente
5		Região após o START não fecha em trincas	BUG - frameshift
6		STOP em fase antes do término esperado	BUG - nonsense / STOP prematuro

Ordem de precedência: uma cascata de decisões



A primeira condição satisfeita determina o resultado. Uma linha com vários problemas recebe o de maior precedência.

Por que o frameshift vem antes do STOP ausente?

A T G G C T A A A C C G G G C

15 bases: as trincas fecham (ATG GCT AAA CCG GGC), mas nenhuma é STOP. **BUG - STOP ausente**

A T G G C T A A A A C C G T A A

16 bases: um A a mais empurra o TAA final para fora do quadro de leitura. **BUG - frameshift**

Arquitetura: um motor puro e uma interface web

Como os dados entram

TXT: uma sequência por linha, sem cabeçalho

CSV: coluna chamada "entrada"

`biocompiler_1.py`

Motor de análise: uma única função, só biblioteca padrão, nada executa no import.

```
validar_sequencia(RNA)  
-> (status, pre_mRNA)
```

Interface web

`app.py` (Streamlit)

Tabela + resumo, relatório detalhado, download em CSV e em TXT.

LER linha

VALIDAR bases

LOCALIZAR START

ANALISAR trincas

LOCALIZAR STOP

CLASSIFICAR

TRANSCREVER
T→U

GERAR relatório

Resumo conceitual do algoritmo (seção 13 da especificação). No código, bases, START e STOP são verificados num único laço sobre a sequência.

O algoritmo: um único laço, cinco flags

```
def validar_sequencia(RNA):
    tam = len(RNA)
    flag1 = True           # bases válidas (caso 2)
    flag2 = False          # START encontrado (caso 3)
    flag3 = False          # STOP encontrado (caso 4/6)
    flag4 = (tam % 3 == 0) # quadro múltiplo de 3 (caso 5)
    flag5 = True           # sem STOP prematuro (caso 6)

    i = 0
    for base in RNA:
        if base not in ['A','T','C','G']:
            flag1 = False
        if not flag2:
            if i >= 2:
                if RNA[i-2:i+1]=='ATG' and (i-2)%3==0:
                    flag2 = True; start = i - 2
            else:
                if RNA[i-2:i+1] in ['TAA','TAG','TGA'] \
                    and (i-2) % 3 == 0:
                    flag3 = True
                    if i + 1 < tam:
                        flag5 = False
        i += 1

    if flag1 and flag2 and flag3 and flag4 and flag5:
        return 'CORRETO', RNA[start:tam].replace('T','U')
    elif not flag1: return 'BUG - base inválida', None
    elif not flag2: return 'BUG - START ausente', None
    elif not flag4: return 'BUG - frameshift', None
    elif not flag3: return 'BUG - STOP ausente', None
    elif not flag5: return 'BUG - nonsense / STOP prematuro', None
```

Passada única

Um só laço testa bases, START e STOP ao mesmo tempo; as flags decidem o diagnóstico no final.

Quadro fixo desde o início da linha

$(i-2) \% 3 == 0$ é contado a partir do caractere 0: ATG e STOP só contam em posições múltiplas de 3.

Frameshift usa o tamanho total

flag4 é $\text{tam} \% 3$, não $(\text{tam} - \text{start}) \% 3$: não depende de onde o START foi achado.

Mesma precedência do slide 7

A ordem dos elif não muda: base inválida > START ausente > frameshift > STOP ausente > nonsense.

Demonstração

```
streamlit run app.py
```

- 1 Enviar entrada.txt (TXT) com as seis sequências da especificação, uma por caso
- 2 Clicar em "Executar validação" e conferir a tabela: entrada, status, pré-mRNA
- 3 Ver o resumo por status e o relatório detalhado logo abaixo
- 4 Baixar resultados em CSV (entrada,status,pre_mRNA)
- 5 Baixar resultados.txt no formato linha;status;resultado;pre_mRNA